CSE 2421

arrays[0]

Java arrays vs C arrays

- Arrays are declared directly with a given size in C rather than using a two-step process of declaring a reference to an array then using the new operator like in Java
- C arrays are not objects. They don’t have methods,
- C arrays don’t know their own size (i.e., there is no array.length method).

Index checking

- Index access is not checked by the compiler
  - Check for valid range manually
  - Especially important for user entered indices
- In all expressions indexing an array, first check the index value against the bounds of the array which were established when the array was defined
- C never performs automatic bounds checking in order to raise speed

```
int x[10];
x[20]= 10;
//will build, unpredictable behavior
```

Index checking

- There's a difference in being outside array bounds but inside your allotted memory; and outside the array bounds and outside your allotted memory!
- sizeof (array) returns the total number of bytes not the index bounds
  - uint32_t x[10];
  - sizeof(x) = 40;
  - //uint32_t is 4 bytes, and there are 10 of them

Initializing Arrays

- Initializing values are enclosed within the curly braces in the declaration and placed following an equal sign after the array name.
- Initialize an individual array location (name[sub]) like any other variable/memory location.

```
int studentAge[4];
studentAge[0]=14;
studentAge[1]=13;
studentAge[2]=15;
studentAge[3]=16;
```

```
//initialize and print all the elements of the array
int myArray [5] = {1,2,3,4,5};
for (int i=0;i<5;i++)
{    printf("%d", myArray[i]);
}
```

If you omit the size of the array, an array just big enough to hold the initialization is created:

```
double balance[] = {1000.0, 2.0, 3.4, 17.0, 50.0};
```

Initializing Arrays

- Initializing all elements to value 0
  - int myarray[10] = {0};
  - static int myarray[10];
    - static variables are automatically 0'd out
  - int myarray[10] = {1,2}; //elements not specified are set to 0, so index 2-9 will = 0
  - memset – see next slide
- If your compiler is GCC you can use following syntax:
  - int array[1024] = {[0 ... 1023] = 5};
    - sets index 0-1023 to 5

Initializing Arrays

- What happens if you give a value that's too big?

```
char temp[4]=“the spoon is too big”
```

compiler will throw a warning, but will build temp[4] = “the “.

void *memset(void *buffer, int ch, size_t count);

- Parameters
  - buffer – Pointer to the buffer you want set
  - ch – byte you want the memory set to
  - count – how many bytes to set
- Returns input parameter buffer

void *memset(void *buffer, int ch, size_t count);

- In library <string.h>

```
void main( void ) {
    char buffer[] = "This is a test of the memset function";
    printf( "Before: %s\n", buffer );
    memset( buffer, '*', 4 );
    printf( "After:  %s\n", buffer ); }
```

- Output:

This is a test of the memset function
**** is a test of the memset function

Copying Arrays

- C does not support deep copy
- copy each item separately into another array.

```
#include <stdio.h>
int main()
{    int iMarks[4] = {78, 64, 66, 74};
     int newMarks[4];
     int i,j;
     for(i=0; i<4; i++)
         newMarks[i]=iMarks[i];
     for(j=0; j<4; j++)
         printf("%d\n", newMarks[j]);
      return 0;
}
```

Array iteration

```
#define SIZE 10
int values[SIZE];
int *vp;
int v_index;

for(v_index = 0; v_index< SIZE;)
    values[v_index++] = 0; //subscript
```

CS 2421

Multidimensional Arrays

Multi-dimensional Arrays

- Declarations – [row][col] subscript order
  - float table [50] [50];
  - char line [24] [40];
  - int values [3] [4] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12 }

		columns			
		0	1	2	3
rows	0	1	2	3	4
	1	5	6	7	8
	2	9	10	11	12

	Column 0	Column 1	Column 2	Column 3
Row 0	a[0][0]	a[0][1]	a[0][2]	a[0][3]
Row 1	a[1][0]	a[1][1]	a[1][2]	a[1][3]
Row 2	a[2][0]	a[2][1]	a[2][2]	a[2][3]

in memory:											
values -->	[0][0]	[0][1]	[0][2]	[0][3]	[1][0]	[1][1]	[1][2]	[1][3]	[2][0]	[2][1]	etc
address -->	+0	+4	+8	+12	+16	+18	etc				

Multi-dimensional Arrays

- MUCH less efficient if traversing array in column order; can do just less efficient than traversing in row order since they are adjacent.

- Locality of Reference:
- “It's typically efficient to access memory which is near other recently accessed memory. This means that the most efficient way to read through a chunk of the array is to vary the rightmost index the most frequently since that will access elements that are near each other in memory.”

Manipulating Arrays Examples

- The array name without any brackets acts as a pointer to the first element

```
int a[ ] = {1,2,3,4};

void intSwap(int *x, int *y)
intSwap(&a[i],&a[n-i-1]);

void printIntArray(int a[], int n)
printIntArray(x,hmny);

int getIntArray(int a[], int nmax, int sentinel)
hmny = getIntArray(x, 10, 0);

void reverseIntArray(int a[], int n)
reverseIntArray(x,hmny);
```

When we pass arrays into functions, the compiler automatically converts the array into a pointer to the first element of the array.
So we just pass the array directly without using the ampersand.

Multi-dimensional Arrays Example

```
#include <stdio.h>
int main()
{
    int x; int y; int array[8][8];  /* Declares an array like a  matrix*/
    for ( x = 0; x < 8; x++ )
    {
        for ( y = 0; y < 8; y++ )
            array[x][y] = x * y;    /* Set each element to a value */
    }
    printf( "Array Indices:\n" );
    for ( x = 0; x < 8;x++ )
    {
        for ( y = 0; y < 8; y++ )
            printf( "[%d][%d]=%d", x, y, array[x][y] );
        printf( "\n" );
    }
    getchar();
    return 0;
}
```

Multi-dimensional Arrays Example

```
int main(void)  {
      char room[100][100];
      print_room(room, height, width)
}

void print_room(char room[100][100], int height, int width)
{
    int i,j;

    for (i=0;i<height;i++) {
          for (j=0;j<width;j++) {
            printf("%c",room[i][j]);
          }
      printf("\n");  // Why do I do this?
    }
}
```

Lab2 Tips

- Quick Demo
- Start now!
- Don’t panic
- Break into pieces and implement individually
  - Reading in and writing out the room (via files)
  - Performing an iteration on puppies only
  - Add rules iteratively
  - ***Make your own test cases***
  - Use functions
- Use GDB
- Debug Printing (dump out the board)
- Ask questions, don't over think the problem